

GSEA - Gestión Segura y Eficiente de Archivos

Examen Final Práctico - Sistemas Operativos

Juan Pablo Rúa Cartagena

Juan José Restrepo Higueta

Fredy Cadavid Franco

Universidad EAFIT

Escuela de Ciencias Aplicadas e Ingeniería

Noviembre 2025

Índice

1. Introducción	3
2. Objetivos	3
3. Tecnologías Utilizadas	3
4. Diseño de la Solución	3
4.1. Arquitectura general	3
4.2. Flujo de datos	4
5. Módulos	4
5.1. FileManager (file_manager.cpp)	4
5.2. Worker (worker.cpp)	4
5.3. HuffmanCompressor (huffman.cpp)	4
5.4. AES (aes.cpp)	4
6. Justificación de algoritmos	5
6.1. Compresión: Huffman	5
6.2. Cifrado: AES-128	5
7. Implementación y consideraciones	5
8. Estrategia de concurrencia	5
9. Guía de uso	5
9.1. Requisitos	5
9.2. Compilación	6
9.3. Ejemplos	6
10.Pruebas	6
11.Trabajo futuro	6
12.Referencias	6

1. Introducción

GSEA (Gestión Segura y Eficiente de Archivos) es una utilidad de línea de comandos desarrollada en C++ para comprimir/descomprimir y encriptar/desencriptar archivos de manera sencilla. Se implementan algoritmos propios de compresión (Huffman) y cifrado simétrico (AES-128 en modo CBC) y se utilizan llamadas directas al sistema operativo para la entrada/salida de archivos.

2. Objetivos

- Implementar compresión/descompresión sin pérdida.
- Implementar cifrado/descifrado simétrico básico.
- Usar llamadas directas al sistema operativo para gestionar archivos.
- Procesar múltiples archivos en paralelo aprovechando concurrencia a nivel de hilo.

3. Tecnologías Utilizadas

- **Lenguaje:** C++17.
- **Plataformas:** Windows (API Win32) y POSIX (Linux, macOS).
- **Concurrencia:** Windows Threads y pthreads.
- **Compilación:** Make / mingw32-make, GCC/Clang/MSVC.

4. Diseño de la Solución

4.1. Arquitectura general

El sistema se organiza en módulos independientes:

- **main.cpp:** Orquestación y rutas de entrada/salida.
- **ArgParser:** Parsing y validación de argumentos.
- **FileManager:** I/O mediante llamadas del SO.
- **Worker:** Procesamiento concurrente (un hilo por archivo de primer nivel).
- **HuffmanCompressor:** Compresión/descompresión.
- **AES:** Cifrado/descifrado AES-128 en modo CBC.

4.2. Flujo de datos

1. El usuario invoca el programa con las opciones deseadas.
2. ArgParser valida operaciones, rutas y clave (si aplica).
3. main.cpp determina si la entrada es archivo o directorio.
4. Para directorios, se listan solo los archivos de primer nivel (no recursivo) y se asigna un hilo a cada uno.
5. Cada hilo lee el archivo, aplica operaciones en orden y escribe la salida.
6. El hilo principal espera a que todos los hilos terminen y reporta el resultado.

5. Módulos

5.1. FileManager (file_manager.cpp)

Realiza la I/O con llamadas nativas:

- **Windows:** CreateFileA, ReadFile, WriteFile, CloseHandle, FindFirstFileA/FindNextFileA.
- **POSIX:** open, read, write, close, opendir/readdir, mkdir.

Se añadió validación de tamaños con GetFileSizeEx (Windows) y verificación de `errno` en POSIX.

5.2. Worker (worker.cpp)

Procesa archivos en paralelo. Crea un hilo por archivo de primer nivel y consolida resultados. No implementa pool de hilos, por lo que directorios con miles de archivos pueden agotar recursos del sistema.

5.3. HuffmanCompressor (huffman.cpp)

Implementa Huffman con min-heap. Se añadió una validación extra para detectar datos comprimidos truncados (comprobación de bits esperados vs. disponibles).

5.4. AES (aes.cpp)

Cifrado/descifrado AES-128 en modo CBC. Validaciones añadidas:

- Rechazo de tamaños de payload que no sean múltiplos del bloque al descifrar.
- Control de mínimo tamaño (requiere IV + al menos un bloque).

Limitaciones actuales: derivación de clave casera (XOR repetido), IV pseudoaleatorio con `std::random_device/mt19937` y sin autenticación de integridad (no hay MAC/AEAD). No debe usarse para seguridad fuerte; se recomienda PBKDF2/scrypt + HMAC o un modo AEAD (GCM/ChaCha20-Poly1305) como trabajo futuro.

6. Justificación de algoritmos

6.1. Compresión: Huffman

Se eligió por ser sin pérdida, bien documentado y eficiente para datos con redundancia. Complejidad: $O(n + k \log k)$ para construir y codificar, con k símbolos únicos. El formato incluye *magic bytes*, tabla de frecuencias y cuenta total de bits, permitiendo reconstrucción del árbol.

6.2. Cifrado: AES-128

Estándar reconocido y eficiente. En este proyecto se implementa modo CBC con padding PKCS#7. Debido a la derivación de clave simplificada y falta de autenticación, el nivel de seguridad efectivo es básico y adecuado solo para fines académicos.

7. Implementación y consideraciones

- El manejo de extensiones ahora elimina correctamente `.aes` y `.huf` cuando se usa `-ud` sobre archivos `.huf.aes`.
- Se valida la longitud de datos cifrados y se detectan bitstreams Huffman truncados.
- La lectura/escritura carga cada archivo completo en memoria; para archivos muy grandes se podría streamizar en trabajo futuro.
- El listado de directorios no es recursivo y el nombre de salida se aplanan (solo nombre de archivo, sin subdirectorios).

8. Estrategia de concurrencia

- Paralelismo a nivel de archivo (un hilo por archivo listado).
- Windows: `CreateThread` + `WaitForMultipleObjects`; POSIX: `pthread_create` + `pthread_join`.
- No hay límite de hilos; recomendable añadir pool para directorios masivos.

9. Guía de uso

9.1. Requisitos

- **Windows:** MinGW-w64 o MSVC con C++17.
- **Linux/macOS:** GCC 7+ o Clang 5+ con pthreads.
- Memoria: suficiente para cargar cada archivo a procesar.

9.2. Compilación

```
1 mingw32-make      # Windows (MinGW)
2 make              # Linux/macOS
```

El ejecutable queda en `bin/gsea.exe` o `bin/gsea`.

9.3. Ejemplos

```
1 bin/gsea -c -i datos.txt          -o datos.huf
2 bin/gsea -d -i datos.huf          -o datos.txt
3 bin/gsea -e -i secreto.pdf         -o secreto.aes -k clave123
4 bin/gsea -ce -i carpeta/           -o salida/      -k clave123  #
   no recursivo
5 bin/gsea -ud -i salida/            -o restaurado/ -k clave123  #
   elimina .aes/.huf
```

10. Pruebas

Se incluyen scripts de prueba (`test_build.sh` y `test_build.bat`) que limpian, compilan y ejecutan un flujo básico de `-c/-d` y `-e/-u` verificando integridad con un archivo de texto repetitivo.

11. Trabajo futuro

- Pool de hilos y procesamiento recursivo de directorios.
- Stream de archivos grandes para reducir uso de memoria.
- Derivación de clave robusta (PBKDF2/scrypt) e inclusión de MAC o modo AEAD.
- Progreso y manejo más férreo de errores E/S.

12. Referencias

1. Huffman, D.A. (1952). “A Method for the Construction of Minimum-Redundancy Codes”.
2. NIST (2001). “FIPS 197: Advanced Encryption Standard (AES)”.
3. Stevens, W.R., & Rago, S.A. (2013). *Advanced Programming in the UNIX Environment*.
4. Russinovich, M.E., Solomon, D.A., & Ionescu, A. (2012). *Windows Internals*.